

Agentic Construction of Diverse High-Level Synthesis Datasets

Tanmay Shukla, Justin Zhou, Kaushik Chandana, Stefan Abi-Karam, Hanqiu Chen, Callie Hao
School of Electrical and Computer Engineering, Georgia Institute of Technology, Atlanta, USA
tshukla31@gatech.edu, justin.zhou@gatech.edu, kchandana6@gatech.edu,
stefanabikaram@gatech.edu, hanqiu.chen@gatech.edu, callie.hao@gatech.edu

Abstract—Machine Learning and large language models (LLMs) for semiconductor design have seen rapid growth across academia and industry, driving new demand for hardware design source code for model training. Most prior dataset efforts in this domain focus on hardware description languages (HDLs). Meanwhile, high-level synthesis (HLS) remains a popular approach for developing domain-specific accelerators and other hardware systems. However, HLS designs are far scarcer than HDL designs, and existing HLS datasets typically emphasize manual code collection or limited implementation data. We propose an agentic two-stage pipeline for constructing the largest community-driven dataset of open-source HLS design. Stage I systematically scrapes publication metadata identifying 2,518 papers and curating those with publicly available HLS artifacts. Stage II deploys mini-SWE-agent—an LLM-backed software engineering agent operating in an isolated Docker environment—to autonomously clone each repository, extract synthesizable kernels, generate testbenches where absent, try compilation, and output all artifacts in a standardized structure.

Preliminary evaluation on open-source repositories shows that the pipeline produces standardized extraction artifacts across heterogeneous repository structures, substantially reducing manual curation effort. Comprehensive compilability and synthesis-rate benchmarking is ongoing, but these initial results establish a reproducible foundation for scaling open HLS datasets for ML/LLM-for-HLS research.

I. INTRODUCTION

Custom IC and ASIC development underpins modern computing, but the tapeout process remains expensive and slow, often requiring long verification loops and high risk. FPGAs provide a faster deployment path by offering the option of reconfigurable hardware, enabling testing to be a more accessible idea. Still, conventional RTL-centric workflows still demand substantial hardware expertise and manual iteration. High-level synthesis (HLS) somewhat mitigates this barrier by synthesizing C/C++ code into RTL and enabling rapid optimization through directives such as pipelining, loop unrolling, and memory partitioning.

Particularly, the higher level of abstraction enables machine learning (ML) and large language model (LLM) methods to be increasingly used for HLS tasks including quality-of-result prediction, design-space exploration, and code generation [1], [2]. These methods are fundamentally data-driven: generalization depends on large, diverse, and validated design sources. However, the HLS data ecosystem remains substantially less mature than its HDL counterpart. In practice, discovering usable open-source HLS designs across repositories is still labor intensive, artifacts are inconsistently formatted, and many

candidate designs cannot be validated end-to-end without significant manual involvement.

This paper addresses this bottleneck through an agentic pipeline for scalable HLS dataset construction. The full implementation is open-sourced at <https://github.com/stefanpie/hlsfactory-agent>. We combine semi-automated source discovery with an LLM-backed software engineering agent to transform heterogeneous repositories into standardized, reusable extraction designs that are directly consumable by downstream benchmarking and synthesis workflows. Our approach is designed to improve reproducibility, reduce curator effort, and expand dataset diversity for modern ML/LLM-for-HLS research.

Our main contributions are:

- **A two-stage, end-to-end data pipeline.** Stage I searches for publication metadata across major scholarly sources to identify candidate HLS repositories at scale; Stage II applies agentic repository analysis and extraction to recover synthesizable design units and associated artifacts.
- **Standardized packaging and validation-oriented outputs.** For each extracted design, the pipeline produces standardized source organization, testbenches, synthesis scripts, manifests, and diagnostic logs, enabling consistent downstream processing.
- **Initial empirical evidence on open repositories.** Preliminary runs on six open-source HLS repositories demonstrate robust artifact extraction across heterogeneous codebases and establish a reproducible foundation for larger-scale compilability and synthesis benchmarking.

The remainder of this paper reviews related work (Section II), details the two-stage methodology, presents preliminary extraction results, and concludes with limitations and future directions.

II. PRIOR WORK

Early ML-for-HLS research relied heavily on benchmark suites such as PolyBench, MachSuite, CHStone, Rosetta, and HLSBench [3]–[7]. These benchmarks enabled foundational progress in performance modeling and optimization, but they were not designed as broad heterogeneous data resources. As a result, they capture only a limited range of kernels and do not reflect the organizational heterogeneity of modern open-source HLS codebases.

More recent dataset efforts improved this landscape. HLSDataset provided a dedicated corpus for ML-assisted HLS/FPGA studies [8], while HLSyn and HLSFactory

advanced large-scale generation, synthesis and standardization workflows [9], [10]. In parallel, LLM-focused HLS research has introduced new benchmarks and workflows for generation, optimization, and evaluation citex-iong2024hlspilot,abikaram2025hlseval,li2025idse.

Despite this progress, prior curation and benchmarking pipelines still face recurring limitations: (1) heavy manual effort to discover and isolate synthesizable HLS kernels from heterogeneous repositories, (2) inconsistent packaging of source, testbench, and synthesis artifacts, and (3) incomplete end-to-end validation of compilability, functional correctness, and synthesis readiness. These limitations reduce reproducibility and restrict the diversity of the data set, motivating an agentic collection approach that can systematically discover, extract, organize, and verify distinct HLS designs at scale.

Our work is most closely related to HLSFactory [10], which provides an open framework for collecting and standardizing HLS designs. We build on that foundation by introducing an agentic pipeline that systematically identifies additional repositories, extracts candidate designs, and outputs validated standardized artifacts suitable for downstream integration.

III. METHODOLOGY

Building on the gaps identified in Section II—manual repository discovery, inconsistent artifact packaging, and limited end-to-end validation—our methodology uses a two-stage pipeline for scalable ML-for-HLS dataset construction. The pipeline first expands the pool of candidate repositories, then converts those heterogeneous codebases into standardized, validation-oriented artifacts.

- **Stage I: Discovery and Curation** of academic HLS sources from publication metadata.
- **Stage II: Agentic Extraction and Validation** of HLS designs using a mini-SWE-agent runtime.

A. Stage I: Automated Discovery and Curation of Academic HLS Sources

1) *Motivation:* Researchers across computer architecture, FPGA design, and electronic design automation (EDA) frequently publish open-source HLS designs to support reproducible experimentation. However, despite widespread use, there is no centralized index of these designs, and relevant artifacts must typically be located manually paper-by-paper. This gap motivates the first component of the methodology: a systematic effort to discover and curate all publicly released HLS designs across major research venues.

2) *Metadata Collection:* To accomplish this, a lightweight Python scraper (<https://github.com/stefanpie/hls-index.git>) was developed using APIs provided by IEEE Xplore, ACM Digital Library, arXiv, and DBLP. The tool collected publication metadata—titles, abstracts, author lists, DOIs, and URLs—and stored them in a spreadsheet for collaborative review. The records are stored in a structured spreadsheet for collaborative triage and auditability.

Papers mentioning HLS in their title or abstract were automatically flagged and then exported into a structured Excel

file for subsequent manual curation. Across all venues, the scraper gathered 2518 papers spanning publication years 1984–2025, representing more than 25 conferences and journals.

3) *Manual Curation and Inclusion Criteria:* Automated metadata filtering is insufficient to guarantee usable code. We therefore manually review flagged records to determine whether each publication links to practical artifacts that can enter an extraction pipeline.

The output of Stage I is a vetted list of repository URLs/paths. This list is the direct input queue for Stage II.

B. Stage II: Agentic Extraction and Validation with mini-SWE-agent

Stage II consumes the repositories found to contain HLS designs and transforms each repository into a structured, synthesis-ready extraction package compatible with HLSFactory. In this context, a *design* refers to a candidate HLS C/C++ accelerator centered on a single synthesizable top-level function.

The central difficulty is that the repositories identified in Stage I are highly heterogeneous: they differ in directory layout, naming conventions, build systems, and dependency assumptions. As noted in Section II, this heterogeneity is a key reason prior curation efforts have relied on heavy manual effort and produced inconsistent packaging of source, testbench, and synthesis artifacts. A fixed rule-based parser cannot reliably generalize across these variations. Stage II addresses this by delegating extraction to an LLM-backed software engineering agent—*mini-SWE-agent* [11]—that can interactively inspect, compile, and adapt to each repository’s differences. Figure 1 illustrates the complete pipeline, from repository input to normalized output, and the remainder of this section walks through each component in detail.

1) *Orchestration and Isolation:* The entry point for a repository extraction run is `main.py`, which coordinates the full lifecycle of a single extraction session. Given an input repository (specified as a GitHub URL or local path), the orchestrator executes the following steps in sequence: it loads environment configuration from a `.env` file (including API credentials), builds the Docker image if one does not already exist, constructs the task prompt that will guide the agent, initializes the agent and executes the extraction session, and exports all produced artifacts and logs to the host output directory.

All agent activity is sandboxed inside a Docker container. The container mounts two key directories—`/workspace` (for repository cloning and intermediate processing) and `/output` (for final artifacts)—and ships with `git`, `curl`, and `clang` pre-installed. Additionally, HLS stub headers are injected into `/workspace/stubs` to resolve vendor-specific includes (e.g., Xilinx `ap_int.h`) that would otherwise cause compilation failures. This containerized design ensures that extraction runs are reproducible, tool-consistent, and free from unintended host-side side effects.

2) *The mini-SWE-agent Runtime:* At the heart of Stage II is *mini-SWE-agent*, an LLM-backed software engineering run-

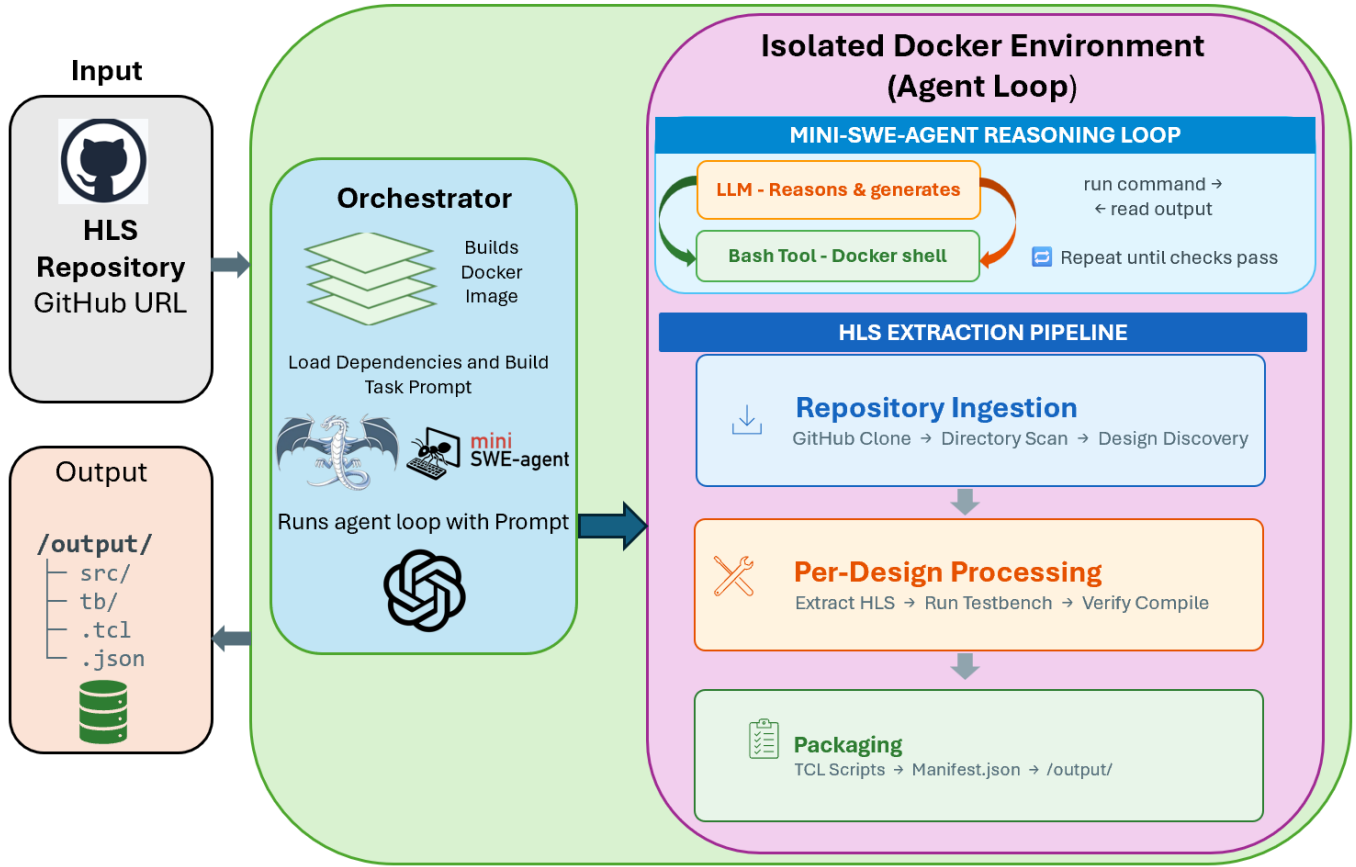


Fig. 1. Stage II workflow for agentic HLS extraction and validation. An input repository (a) is processed by the orchestrator (b), which launches *mini-SWE-agent* inside an isolated Docker environment (c). The agent iteratively clones, analyzes, extracts, and validates HLS designs, producing a normalized output package (d) that is directly ingestible by the HLSFactory dataset.

time that serves as the extraction workforce. The implementation instantiates a `DefaultAgent` configured with two components: a language model backend (`OpenRouterModel` with `model_name="moonshotai/kimi-k2.5"`) and a sandboxed execution environment (`DockerEnvironment` targeting the image).

The agent operates through an iterative *reason-act-observe* loop. In each iteration, the LLM inspects the current state of the repository (e.g., directory structure, file contents, or compiler output), reasons about the next action to take, emits a concrete shell command, and then observes the result of that command. Based on the observed output—which may include filesystem changes, compiler diagnostics, or error messages—the agent refines its strategy and proceeds to the next iteration. This loop repeats until the agent determines that all extraction checks have passed or that no further progress can be made.

This adaptive, feedback-driven approach is what distinguishes Stage II from a static script. Where a rule-based extractor would fail on an unexpected directory layout or a missing header file, the agent can diagnose the issue from compiler output and attempt a targeted fix, such as locating the correct include path or generating a missing dependency

stub.

3) *Prompt-Driven Extraction Workflow*: The agent’s behavior within each session is governed by a structured prompt constructed in `build_prompt()`. This prompt encodes the complete extraction workflow as a set of high-level instructions, which the agent executes autonomously. The workflow proceeds in three phases:

a) *Repository Ingestion*.: The agent begins by cloning the target repository into `/workspace/repo` and analyzing its structure. It traverses the file tree to identify candidate HLS designs—typically C/C++ source files containing synthesizable top-level functions annotated with HLS pragmas or associated with HLS build configurations.

b) *Per-Design Processing*.: For each identified design, the agent creates a dedicated directory under `/output/<repo_name>/` and populates it with the relevant source files. It resolves local dependencies (e.g., shared header files or utility functions) by tracing includes and copying the necessary files. If a testbench is already present in the repository, the agent collects it alongside the source; otherwise, it generates a minimal testbench to ensure the design can be compiled and exercised in isolation. The agent then compiles the design using `clang` as a lightweight syntax

and type check, recording any diagnostics in a per-design `compile_log.txt`. In parallel, it generates documentation (a README describing the design) and a Vitis HLS-compatible TCL synthesis script (`run_hls.tcl`).

c) *Packaging.*: Once all designs have been processed, the agent organizes them into the following normalized directory structure:

```
output/<repo_name>/
  <design_name>/
    source files (.cpp, .h)
    testbench (.cpp)
    run_hls.tcl
    README.md
    compile_log.txt
    ...
  manifest.json
```

A repository-level `manifest.json` file summarizes all extracted designs, their compilation status, and associated metadata. This normalization strips away repository-specific assumptions about paths, build systems, and naming conventions, enabling uniform downstream processing by HLSFactory.

4) *Validation Criteria*: The three limitations identified in Section ??—manual kernel isolation, inconsistent artifact packaging, and incomplete end-to-end validation—motivate the explicit validation targets that Stage II enforces for each extracted design:

- 1) **Compilability**. Each design must compile successfully under a standalone `clang` invocation with the provided stub headers. This check catches missing includes, syntax errors, and type mismatches before the design enters the dataset.
- 2) **Functional readiness**. Every design must be accompanied by a testbench—either discovered in the source repository or generated by the agent—so that it can be exercised independently of its original build environment.
- 3) **Synthesis readiness**. Each design must include a TCL script and sufficient metadata to be fed directly into a Vitis HLS synthesis flow without manual intervention.

Importantly, extraction does not silently discard designs that fail these checks. When compilation errors or other issues persist after the agent’s repair attempts, all diagnostic artifacts (e.g., `compile_log.txt`) are preserved alongside the design. This deliberate retention of failure cases enables systematic post-hoc error analysis and supports iterative refinement of both extraction prompts and validation logic.

5) *Telemetry and Reproducibility*: Each extraction run produces structured telemetry alongside the design artifacts. The full agent trajectory—the complete sequence of reasoning steps, shell commands, and their outputs—is stored in a `*.traj.json` file. A companion `*_benchmark.json` file records run-level metadata: a unique run identifier, timestamp, repository URL, model configuration, exit status, total API call count, estimated cost, and the trajectory filename.

Together, these records serve three purposes: they enable exact reproduction of any individual extraction run, they support quantitative evaluation of extraction robustness across the full corpus of repositories, and they provide the data needed for ongoing improvement of prompt design and validation policies.

IV. RESULTS

We have currently run the extraction pipeline on 6 open-source HLS repositories. These initial runs used OpenCode as its agent, but the pipeline has since been migrated to mini-SWE-agent, which provides structured JSON telemetry. As a result, detailed metrics such as LLM call count, token usage, and API cost are not yet available for the evaluated repositories, but will be captured for all future runs.

Across the 6 repositories, the agent successfully scanned, extracted, and organized content to produce standardized output directories with source files, testbenches, TCL scripts, and READMEs. We noticed that some repositories initially failed extraction or only produced partial outputs, but re-running the agent on failed repositories is a planned next step. Validation of the correctness of these directories using `clang` and synthesis in Vitis HLS has not yet been performed.

Full evaluation is ongoing and will include how many designs the agent successfully extracts per repository, `clang` compilation pass rate, and Vitis HLS synthesis results. Agent efficiency metrics from the mini-SWE-agent JSON logging, including LLM call count, token usage, and estimated API cost, will be aggregated across all future runs to determine pipeline scalability and cost. Once validation for each repository is complete, the extracted designs will be integrated into HLSFactory.

V. CONCLUSION

We present an open-source agentic pipeline for scalable HLS design extraction, addressing a core bottle-neck in ML/LLM-for-HLS research: a lack of large, diverse, and consistently validated design corpora. By combining automated source discovery with the LLM-backed software engineering agent *mini-SWE*, our two-stage workflow successfully extracts, validates, and packages HLS designs from heterogeneous GitHub repositories— with minimal manual intervention. All extracted designs from the pipeline will be added to HLSFactory, directly enhancing its dataset coverage and comprehensiveness. The full pipeline has been open-sourced at (<https://github.com/stefanpie/hlsfactory-agent>) enabling the community to reproduce our results, expand the corpus, and extract new HLS designs from other repositories, further contributing to an ecosystem more suitable for ML/LLM-for-HLS research.

ACKNOWLEDGEMENT

This work was supported by the Opportunity Research Scholars’ Program in the School of Electrical and Computer Engineering at the Georgia Institute of Technology.

REFERENCES

- [1] A. Sohrabizadeh, Y. Bai, Y. Sun, and J. Cong, “Automated accelerator optimization aided by graph neural networks,” in *Proceedings of the 59th ACM/IEEE Design Automation Conference (DAC ’22)*, 2022, pp. 55–60. [Online]. Available: <https://dl.acm.org/doi/10.1145/3490422.3502330>
- [2] M. Gao, J. Zhao, Z. Lin, and M. Guo, “Hierarchical source-to-post-route qor prediction in high-level synthesis with gnns,” *arXiv preprint arXiv:2401.08696*, Jan. 2024, available: <https://arxiv.org/abs/2401.08696>.
- [3] L.-N. Pouchet, “Polybench: The polyhedral benchmark suite,” <http://www.cs.ucla.edu/~pouchet/software/polybench/>, 2012.
- [4] B. Reagen, R. Adolf, Y. S. Shao, G.-Y. Wei, and D. Brooks, “Machsuite: Benchmarks for accelerator design and customized architectures,” in *2014 IEEE International Symposium on Workload Characterization (IISWC)*, 2014, pp. 110–119.
- [5] Y. Hara, H. Tomiyama, S. Honda, and H. Takada, “Proposal and quantitative analysis of the chstone benchmark program suite for practical c-based high-level synthesis,” *Journal of Information Processing*, vol. 17, pp. 242–254, Oct. 2009.
- [6] Y. Zhou, U. Gupta, S. Dai, R. Zhao, N. Srivastava, H. Jin, J. Featherston, Y.-H. Lai, G. Liu, G. A. Velasquez, W. Wang, and Z. Zhang, “Rosetta: A realistic high-level synthesis benchmark suite for software programmable fpgas,” in *Proceedings of the 2018 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, ser. FPGA ’18, 2018, pp. 269–278.
- [7] N. Kavvadias and K. Masselos, “Automated synthesis of fsmd-based accelerators for hardware compilation,” in *2012 IEEE 23rd International Conference on Application-Specific Systems, Architectures and Processors (ASAP)*, Delft, Netherlands, 2012, pp. 157–160.
- [8] X. Wei *et al.*, “Hlsdataset: A benchmark suite for machine learning-based high-level synthesis optimization,” in *Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*, 2023.
- [9] Y. Bai *et al.*, “Hlsyn: Towards large-scale hls benchmark generation and evaluation,” in *ACM/IEEE Design Automation Conference (DAC)*, 2023.
- [10] S. Abi-Karam, R. Sarkar, A. Seigler, S. Lowe, Z. Wei, H. Chen, N. Rao, L. John, A. Arora, and C. Hao, “HLSFactory: A Framework Empowering High-Level Synthesis Datasets for Machine Learning and Beyond,” in *2024 ACM/IEEE 6th Symposium on Machine Learning for CAD (MLCAD)*. Salt Lake City (Snowbird), UT, USA: IEEE, Sep. 2024, pp. 1–9.
- [11] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. R. Narasimhan, and O. Press, “SWE-agent: Agent-computer interfaces enable automated software engineering,” in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. [Online]. Available: <https://arxiv.org/abs/2405.15793>